

Phần 2 - Design Principles (SOLID)

Mục lục

2.1 Tổng quan Phần 2: Design Principles (SOLID)	4
Vì sao phần này đứng thứ hai	5
Nếu bạn đến từ Data Science	5
Phần 2 sẽ dạy gì?	5
Phần A: Foundation (1 bài)	5
Phần B: Năm nguyên lý SOLID (5 bài)	5
Vì sao SOLID quan trọng cho architect	6
Lý do 1: Architect phải review design của team	6
Lý do 2: SOLID là building block cho architecture patterns lớn hơn	6
Lý do 3: Refactoring legacy là việc architect hay phải định hướng	6
SOLID không miễn phí	6
Thứ tự đọc	7
Kết nối với các phần sau	7
2.2 Cohesion và Coupling	8
Nếu bạn đến từ Data Science	8
Một câu hỏi trước khi vào định nghĩa	8
Cohesion là gì?	8
1. Coincidental cohesion (tệ nhất)	8
2. Logical cohesion (tệ)	9
3. Temporal cohesion (trung bình)	9
4. Procedural cohesion (trung bình)	9
5. Communicational cohesion (khá)	9
6. Sequential cohesion (tốt)	9
7. Functional cohesion (tốt nhất)	10
Quy tắc thực dụng	10
Coupling là gì?	10
1. Data coupling (nhẹ nhất, tốt)	10
2. Stamp coupling	10
3. Control coupling	11
4. External coupling	11
5. Common coupling	11
6. Content coupling (tệ nhất)	11
Quy tắc thực dụng	11
Mục tiêu vĩnh cửu: High cohesion + Low coupling	12
Lý do 1: Cost-to-understand	12
Lý do 2: Cost-to-change	12

Lý do 3: Testability	12
Lý do 4: Reusability	12
Quan hệ giữa Cohesion-Coupling và SOLID	12
Khi nào “coupling thấp” lại là bẫy?	13
Tóm tắt	13
2.3 SRP, Single Responsibility Principle	14
Nếu bạn đến từ Data Science	14
Phiên bản hay bị giảng sai	14
Định nghĩa modern: SRP về Actor	14
Ví dụ kinh điển	15
Vi phạm SRP	15
Tuân thủ SRP	15
Cách identify actor trong dự án	16
Heuristic 1: Hỏi “ai sẽ yêu cầu sửa cái này?”	16
Heuristic 2: Map class lên org chart	16
Heuristic 3: Theo dõi history	16
Heuristic 4: Theo dõi merge conflict	16
Các pattern fix SRP violation	17
Pattern 1: Extract Class	17
Pattern 2: Facade pattern	17
Pattern 3: Strategy pattern (cho variation theo actor)	17
Khi nào SRP là over-engineering	18
Câu 1: Có thực sự 2+ actor không?	18
Câu 2: Lifespan dự án bao lâu?	18
Câu 3: Có merge conflict không?	18
Sai lầm thường gặp	18
Sai lầm 1: “Một method = một class”	18
Sai lầm 2: Coi “module” = “class”	18
Sai lầm 3: Coi “responsibility” = “task”	18
Sai lầm 4: Bỏ qua context	18
SRP ở mức Architecture	18
Tóm tắt	19
2.4 OCP, Open-Closed Principle	20
Nếu bạn đến từ Data Science	20
Định nghĩa	20
Ví dụ thuyết phục	20
Vi phạm OCP	20
Tuân thủ OCP	21
Cơ chế thường dùng	22
1. Polymorphism qua interface/abstract class	22
2. Strategy pattern	22
3. Plugin architecture (Microkernel)	22
4. Hook / Callback	22
5. Configuration	22
OCP ở mức Architecture	22
Microkernel (Plug-in) Architecture	22
Event-Driven Architecture	23

API Versioning	23
Configuration-driven systems	23
Khi nào OCP là over-engineering	23
Sai lầm thường gặp	24
Sai lầm 1: Coi mọi if-elif là OCP violation	24
Sai lầm 2: Abstract mọi class	24
Sai lầm 3: Quên invariant của abstraction	24
Sai lầm 4: OCP làm hỏng cohesion	24
Tóm tắt	24
2.5 LSP, Liskov Substitution Principle	26
Nếu bạn đến từ Data Science	26
Phát biểu gốc	26
Ví dụ kinh điển: Square is-a Rectangle?	26
Cách sửa	27
Contract của abstraction: ba thành phần	27
1. Preconditions	28
2. Postconditions	28
3. Invariants	28
Liskov's rule chính thức	28
Ví dụ trong thực tế	28
Vi phạm 1: Square / Rectangle (đã xem)	28
Vi phạm 2: Penguin / Bird	28
Vi phạm 3: Read-only override	29
Vi phạm 4: ORM "lazy" overriding	29
Code smell báo hiệu LSP violation	29
Smell 1: if isinstance(x, SubType) trong client code	29
Smell 2: Override để throw NotImplementedError	29
Smell 3: Override để do-nothing	30
Smell 4: Tham số override với kiểu hẹp hơn	30
LSP ở mức Architecture	30
REST API versioning	30
Database migration backward compatibility	30
Service interface stability	30
Plug-in interface	30
Sai lầm thường gặp	30
Sai lầm 1: Tin compiler đã check LSP	30
Sai lầm 2: Coi LSP chỉ là về inheritance	31
Sai lầm 3: Cố ép is-a relationship khi nó không có	31
Sai lầm 4: Bỏ qua non-functional contract	31
Tóm tắt	31
2.6 ISP, Interface Segregation Principle	32
Nếu bạn đến từ Data Science	32
Định nghĩa	32
Ví dụ cổ điển	32
Vi phạm ISP	32
Tuân thủ ISP	32
ISP ở mức code	33

Pattern 1: Role-based interfaces	33
Pattern 2: Capability interfaces	33
Pattern 3: Tách theo client	33
ISP ở mức Architecture	34
BFF, Backend for Frontend pattern	34
API Gateway pattern	34
CQRS, Command Query Responsibility Segregation	34
Khi nào ISP là over-engineering	35
Sai lầm thường gặp	35
Sai lầm 1: Single-method interfaces khắp nơi	35
Sai lầm 2: Coi ISP đồng nghĩa với “interface nhỏ”	36
Sai lầm 3: Quên client là ai	36
Sai lầm 4: Confused với SRP	36
Tóm tắt	36
2.7 DIP, Dependency Inversion Principle	37
Nếu bạn đến từ Data Science	37
Phát biểu	37
Direction tự nhiên (sai) vs Inverted (đúng)	37
Direction tự nhiên: top-down	37
Direction đúng: inverted	38
Vì sao gọi là “Dependency Inversion”?	38
Lợi ích cụ thể	39
1. Test nhanh, isolated	39
2. Đổi infrastructure dễ	39
3. Plug-in architecture	40
4. Parallel development	40
DIP và Dependency Injection (DI)	40
Ba kiểu DI	40
DI Container / IoC Container	40
DIP ở mức Architecture	41
Hexagonal / Clean / Onion Architecture	41
Microservices boundary	41
Plug-in System	41
Event-driven systems	42
Sai lầm thường gặp	42
Sai lầm 1: Tạo interface “phòng ngừa” cho mọi class	42
Sai lầm 2: Service Locator anti-pattern	42
Sai lầm 3: Lạm dụng abstract base class	42
Sai lầm 4: DIP nhưng abstraction leak detail	43
Tóm tắt	43
Tổng kết Phần 2	43

2.1 Tổng quan Phần 2: Design Principles (SOLID)

Tóm tắt một dòng: Kiến trúc tốt luôn bắt đầu từ code tốt, và “code tốt” có ý nghĩa rất cụ thể: 5 nguyên lý SOLID + nguyên tắc cohesion cao - coupling thấp. Master 7 bài trong phần này là bạn đã có nền cho mọi phần sau.

Vì sao phần này đứng thứ hai

Bạn có thể hỏi: “Khoá tên là Software Architecture, sao bài đầu lại nói về principles cấp module?”. Câu trả lời nằm ở một metaphor cổ điển của Robert C. Martin:

“Good software systems begin with clean code. If the bricks aren’t well made, the architecture of the building doesn’t matter much. On the other hand, you can make a substantial mess with well-made bricks. This is where the SOLID principles come in.”

Nghĩa là:

- **Bricks không tốt** □ architecture không cứu được. Class hierarchy mục nát thì dù bạn dùng microservices hay layered, mỗi service vẫn là một đống bug.
- **Bricks tốt nhưng xếp sai** □ vẫn ra một đống lộn xộn. Bạn có class tuân thủ SOLID nhưng tổ chức module sai thì hệ thống vẫn khó maintain.

SOLID đảm bảo bricks tốt. Architecture (Phần 3-7) đảm bảo xếp đúng. Cả hai đều cần, và SOLID phải đến trước vì nó là tiền đề.

Nếu bạn đến từ Data Science

Với Data Scientist, SOLID có thể nghe như một chủ đề thuần OOP, xa lạ với notebook và pipeline. Nhưng hãy nhìn vào một tình huống rất quen: một notebook training ban đầu chỉ vài cell, sau đó lớn dần thành 2000 dòng. Nó vừa đọc dữ liệu, clean, join, feature engineering, train model, tune hyperparameter, evaluate, save artifact, tạo chart và export report. Mỗi lần sửa một bước nhỏ, bạn sợ làm hỏng bước khác.

Đó chính là vấn đề SOLID muốn giải quyết ở cấp code. SRP giúp tách `data_loader`, `feature_builder`, `trainer`, `evaluator`. OCP giúp thêm model mới mà không sửa `evaluator`. LSP giúp mọi model tuân thủ cùng `contract fit/predict`. ISP giúp không ép model nào cũng phải có `explain()` nếu nó không hỗ trợ. DIP giúp training logic không phụ thuộc cứng vào S3, BigQuery hay MLflow.

Nói cách khác: SOLID là cây cầu từ notebook prototype sang package ML production có thể test, review và maintain.

Phần 2 sẽ dạy gì?

Phần này có 7 bài, chia hai phần:

Phần A: Foundation (1 bài)

Bài 2.2: Cohesion và Coupling, định nghĩa hai khái niệm cốt lõi nhất trong thiết kế modular. Cohesion đo độ “gắn kết” của các phần tử *bên trong* một module; Coupling đo độ “ràng buộc” *giữa* các module. Mục tiêu vĩnh cửu: **high cohesion + low coupling**. Toàn bộ SOLID có thể được suy ra từ hai nguyên tắc này, nhưng SOLID cho operational guidance cụ thể hơn.

Phần B: Năm nguyên lý SOLID (5 bài)

Bài 2.3: SRP, Single Responsibility Principle, “Mỗi module nên có duy nhất một lý do để thay đổi”. Đây là principle khó nhất để hiểu đúng (thường bị giảng sai). Bài này dùng định nghĩa modern của Robert Martin: SRP nói về *actors* chứ không phải *functions*.

Bài 2.4: OCP, Open-Closed Principle, “Open for extension, closed for modification”. Module nên mở rộng được khả năng mà không phải sửa code đã có. Cách phổ biến: dùng interface/abstraction + polymorphism. Plug-in architecture là OCP ở mức kiến trúc.

Bài 2.5: LSP, Liskov Substitution Principle, Subtype phải thay thế được supertype mà không phá behavior của client. Bài này giải thích vì sao “Square is-a Rectangle” là sai trong code, và cách dùng contract design để check LSP.

Bài 2.6: ISP, Interface Segregation Principle, Client không nên depend vào interface chứa method nó không dùng. Hệ quả: thà nhiều interface nhỏ hơn một interface to. Apply ở mức architecture: API gateway pattern, BFF (Backend-for-Frontend).

Bài 2.7: DIP, Dependency Inversion Principle, High-level module không depend vào low-level module; cả hai depend vào abstraction. Đây là principle quan trọng nhất ở mức architecture, làm nền cho hexagonal/clean architecture và dependency injection framework.

Vì sao SOLID quan trọng cho architect

Có một misunderstanding phổ biến: “SOLID là principles cho developer, architect không cần quan tâm”. Sai. SOLID đặc biệt quan trọng cho architect vì ba lý do:

Lý do 1: Architect phải review design của team

Một architect không hands-on review code, design proposal, và pull request thì kiến trúc bạn vẽ trên giấy sẽ bị implement sai. SOLID là language chung để feedback design, không có nó, feedback của architect mãi ở mức “tôi thấy không ổn, sửa đi” mà không nói rõ vì sao.

Lý do 2: SOLID là building block cho architecture patterns lớn hơn

- Hexagonal architecture xuất phát từ DIP.
- Plug-in architecture (Microkernel style) là OCP ở scale lớn.
- Microservices được justify bằng SRP ở mức service.
- API gateway pattern xử lý đúng vấn đề ISP đặt ra.
- Strategy pattern, Adapter pattern, Decorator pattern, đều là cách áp dụng SOLID.

Không nắm SOLID, bạn học pattern lớn sẽ chỉ thuộc *cái gì* mà không hiểu *vì sao*.

Lý do 3: Refactoring legacy là việc architect hay phải định hướng

Khi team kế thừa codebase 5-10 năm tuổi, architect thường được hỏi: “Nên refactor cái gì trước?”. SOLID violations là một heuristic tốt để rank technical debt, class vi phạm nặng SRP/OCP/DIP thường là nơi bug tập trung và cost-to-change cao nhất.

SOLID không miễn phí

Cảnh báo trước khi bạn vào phần B: SOLID có *cost*, không phải “lúc nào cũng đúng”. Áp dụng SOLID đặt ra:

- Thêm interfaces, abstractions □ tăng số file, tăng cognitive load lúc đọc.
- Tăng indirection □ debug khó hơn (phải jump qua nhiều layer).
- Risk over-engineering □ một startup MVP áp SOLID toàn diện thường delay 2-3x.

Một nguyên tắc empirical:

- **Apply SOLID gần như mặc định** trong code production có lifespan > 2 năm và team > 5 người.
- **Apply selectively** trong startup MVP, code prototype, hoặc script chạy một lần.
- **Đo bằng pain**: chỗ nào sửa thường xuyên gây bug □ áp SOLID trước; chỗ nào ổn định không sửa □ để yên.

Phần 3 (Architectural Thinking) sẽ build lên insight này với khái niệm “iatrogenic” (bệnh do thầy thuốc gây ra), over-engineering cũng là một loại bệnh.

Thứ tự đọc

Đề nghị đọc theo thứ tự tự nhiên: 2.2 □ 2.3 □ 2.4 □ 2.5 □ 2.6 □ 2.7. Mỗi bài khoảng 4000-5000 chữ, đọc kỹ 1.5-2h.

Nếu bạn đã quen với SOLID từ trước, có thể skip 2.2 (cohesion-coupling) nhưng vẫn nên đọc tuần tự 5 bài SOLID vì cách trình bày trong tài liệu này có thể khác với cách bạn đã học. Đặc biệt 2.3 (SRP) thường bị giảng sai, đáng đọc lại.

Bài kế thúc phần: hết Phần 2, bạn sẽ vào [Phần 3: Architectural Thinking](#), mở rộng tư duy từ class lên hệ thống.

Kết nối với các phần sau

- **Phần 3 (Modularity)** dùng cohesion-coupling từ Bài 2.2 ở mức module/sub-system.
- **Phần 4 (Quality Attributes)** liên hệ “modifiability”, một QA quan trọng, với SOLID violations.
- **Bài 5.5 (Microkernel)** chính là OCP ở scale lớn.
- **Bài 6.3 (Microservices)** justify bằng SRP ở mức service.
- **Bài 7.2 (Module Views)** vẽ ra structure mà SOLID giúp tạo.

Hãy bắt đầu với [Bài 2.2: Cohesion và Coupling](#).

2.2 Cohesion và Coupling

Tóm tắt một dòng: Một module tốt là module mà các phần tử bên trong nó tập trung vào cùng một mục đích (cohesion cao) và không bị ràng buộc chặt vào các module khác (coupling thấp). Hai chỉ số này quyết định cả cost-to-change và cost-to-understand của codebase.

Nếu bạn đến từ Data Science

Cohesion và coupling xuất hiện rất rõ trong notebook. Một notebook có low cohesion khi cùng một file vừa load data, vừa clean, vừa train, vừa plot, vừa export report. Một notebook có high coupling khi đổi tên một column ở bước feature engineering làm vỡ training, evaluation và dashboard ở nhiều cell khác nhau.

Khi chuyển sang production, mục tiêu là tách trách nhiệm: data loading, validation, feature transform, training, evaluation, serving, monitoring. Mỗi phần nên có cohesion cao và giao tiếp với phần khác qua contract rõ. Nếu làm được điều này, pipeline dễ test hơn, model dễ thay hơn, và lỗi data dễ khoanh vùng hơn.

Một câu hỏi trước khi vào định nghĩa

Bạn được giao maintain một codebase mới. Hai ngày sau, sếp yêu cầu thêm tính năng X. Bạn mở module dự định sửa và thấy nó:

- Có 47 method, trong đó chỉ 3 method liên quan tới X. 44 method còn lại là validation, logging, email sending, PDF export, tax calculation..., nói chung là “tất cả mọi thứ”.
- Để hiểu logic của 3 method liên quan, bạn phải đọc 5 module khác mà nó import, mỗi module lại depend vào 3-5 module khác.
- Sau 4 giờ đọc code, bạn vẫn không chắc sửa method có gây side effect ở chỗ nào không.

Codebase trên có cohesion thấp (module ôm đồm) và coupling cao (đụng đến 1 thứ phải hiểu 10 thứ). Đó là code khó maintain, không phải vì developer dở, mà vì *structure* sai. Cohesion và coupling là hai chỉ số quan trọng nhất để chẩn đoán “structure sai” ở mức module.

Cohesion là gì?

Cohesion đo *mức độ các phần tử bên trong một module tập trung vào một mục đích duy nhất*. Module có cohesion cao là module mà mọi method/field/class bên trong nó cùng phục vụ một bài toán cụ thể. Module có cohesion thấp là module “tạp hoá”: chứa mọi thứ không liên quan nhau.

Constantine và Yourdon (1979) phân cohesion thành 7 mức, từ thấp nhất tới cao nhất:

1. Coincidental cohesion (tệ nhất)

Các phần tử ở cùng module hoàn toàn ngẫu nhiên, không có lý do logic nào. Ví dụ: một file `utils.py` chứa `parse_date`, `send_email`, `calculate_tax`, `compress_image`. Chúng không liên quan nhau, chỉ “tiện” đặt cùng chỗ.

```
# Coincidental cohesion - tệ
class Utils:
    def parse_date(self, s): ...
    def send_email(self, to, msg): ...
    def calculate_tax(self, amount): ...
    def compress_image(self, img): ...
```


2. Logical cohesion (tệ)

Các phần tử được nhóm vì “cùng loại” hoạt động, nhưng làm việc khác nhau. Thường thấy ở các class kiểu `InputHandler` xử lý mọi input nhưng mỗi input là một logic riêng:

```
class InputHandler:
    def handle(self, kind, data):
        if kind == 'json':
            return json.loads(data)
        elif kind == 'xml':
            return self._parse_xml(data)
        elif kind == 'csv':
            return self._parse_csv(data)
        # ... 20 nhánh khác
```

Vấn đề: thêm format mới phải sửa class này, vi phạm OCP. Cohesion logical thường được fix bằng polymorphism.

3. Temporal cohesion (trung bình)

Các phần tử ở cùng module vì *cùng thời điểm chạy*, không phải vì *cùng mục đích*. Ví dụ điển hình: `init()` function gọi 20 việc khởi tạo khác nhau (`init logger`, `init DB`, `init cache`, `init metrics`, ...).

```
def initialize():
    setup_logging()
    connect_database()
    warm_cache()
    register_metrics()
    load_config()
```

Acceptable cho startup script, nhưng không nên dùng làm pattern cho business logic.

4. Procedural cohesion (trung bình)

Các phần tử cùng module vì cùng *thuộc một thủ tục*, chạy nối tiếp nhau. Ví dụ: `process_order` gọi `validate_input` → `check_stock` → `reserve_inventory` → `charge_card` → `ship`. Mỗi step không liên quan trực tiếp tới step trước về mặt data, nhưng cùng thuộc một workflow.

5. Communicational cohesion (khá)

Các phần tử cùng module vì cùng *thao tác trên một loại data*. Ví dụ: một class `OrderReport` có `generate_csv()`, `generate_pdf()`, `generate_excel()`, tất cả cùng tạo report từ `Order` entity.

6. Sequential cohesion (tốt)

Các phần tử cùng module vì *output của cái này là input của cái kia*. Ví dụ: pipeline xử lý ảnh: `load` → `resize` → `grayscale` → `save`, mỗi step nhận output của step trước.

7. Functional cohesion (tốt nhất)

Mọi phần tử cùng module hợp tác để *làm đúng một việc, không gì khác*. Class `TaxCalculator` chỉ tính thuế, load data, save data, format output đều ở module khác.

```
class TaxCalculator:
    """Tính thuế theo luật Việt Nam 2024."""

    def __init__(self, brackets: list[TaxBracket]):
        self._brackets = brackets

    def calculate(self, income: Money) -> Money:
        # Logic tính thuế duy nhất, không làm việc gì khác
        ...
```

Quy tắc thực dụng

Nhớ 7 mức trên là cứng. Để nhớ hơn là **3 mức**:

Mức	Đặc điểm	Action
Thấp (coincidental, logical)	Module “tạp hoá”	Tách ra
Trung bình (temporal, procedural)	Có lý do gom chung nhưng yếu	OK cho infrastructure
Cao (communicational, sequential, functional)	Có lý do mạnh, nhất quán	Target cho business logic

Coupling là gì?

Coupling đo *mức độ một module bị ràng buộc vào module khác*. Hai module coupling cao là hai module mà thay đổi một bên thường buộc bên kia phải sửa. Hai module coupling thấp là hai module có thể evolve độc lập.

Coupling cũng được Constantine-Yourdon phân thành nhiều mức, từ nhẹ nhất tới nặng nhất:

1. Data coupling (nhẹ nhất, tốt)

Module A gọi module B và chỉ truyền data đơn giản (primitive type hoặc DTO).

```
result = calculator.calculate(income=100_000_000, year=2024)
```

A không biết B implement ra sao. Chỉ data đi qua boundary.

2. Stamp coupling

A truyền cho B một struct/object lớn nhưng B chỉ dùng một phần.

```
def calculate_tax(self, user: User): # nhận cả User
    return user.income * 0.1 # nhưng chỉ dùng .income
```

Vấn đề: A bị couple với toàn bộ schema của User, dù chỉ cần income. Nếu User đổi schema, A có thể vỡ.

3. Control coupling

A truyền cho B một flag điều khiển flow.

```
def render(self, mode: str):
    if mode == 'fast':
        ...
    elif mode == 'high_quality':
        ...
```

A biết “có 2 mode” □ A và B đều cần đồng bộ. Thường refactor thành 2 method riêng hoặc polymorphism.

4. External coupling

Hai module cùng dùng một format ngoài (vd: file format, network protocol). Đổi format □ cả hai phải sửa cùng lúc.

5. Common coupling

Hai module cùng share global mutable state.

```
# config.py
DB_CONNECTION = None # global

# module_a.py
config.DB_CONNECTION = create_pool()

# module_b.py
config.DB_CONNECTION.execute(...) # depend on global
```

Đổi cách init DB □ mọi module dùng DB_CONNECTION đều có thể vỡ. Test khó vì state share.

6. Content coupling (tệ nhất)

Một module truy cập *trực tiếp internal* của module khác, gọi private method, đọc private field qua reflection, monkey-patch class của module khác.

```
# module_b.py
class Cache:
    def __init__(self):
        self._store = {} # private

# module_a.py
cache._store['key'] = 'value' # dùng trực tiếp internal
```

Đổi internal của B □ A vỡ ngay. Tránh tuyệt đối.

Quy tắc thực dụng

Tương tự cohesion, có thể gom thành **3 mức**:

Mức	Loại	Action
Nhẹ (data, stamp)	OK	Default state
Trung bình (control, external)	Refactor được thì refactor	Spot và fix dần
Nặng (common, content)	Phải xử ngay	Refactor priority cao

Mục tiêu vĩnh cửu: High cohesion + Low coupling

Hai nguyên tắc này là *kim chỉ nam* cho mọi quyết định module design. Tại sao?

Lý do 1: Cost-to-understand

Cohesion cao □ mở module ra bạn hiểu ngay nó làm gì. Đọc một file `TaxCalculator.py` 200 dòng tập trung vào tính thuế dễ hơn nhiều đọc file `BusinessLogic.py` 200 dòng làm đủ thứ.

Coupling thấp □ để hiểu module A, bạn không cần đọc 10 module khác. Self-contained.

Lý do 2: Cost-to-change

Cohesion cao □ sửa một thứ chỉ cần sửa một chỗ. SRP nói cùng ý: “Mỗi module nên có một lý do thay đổi”.

Coupling thấp □ sửa module A không lan ra module B, C, D. Bug fix hay feature add đều khoanh vùng được.

Lý do 3: Testability

Module cohesion cao □ test focus vào một concern, không cần setup nhiều thứ.

Module coupling thấp □ mock dependencies dễ. Coupling cao □ mock nhiều, test mất công.

Lý do 4: Reusability

Module cohesion cao + coupling thấp = reusable. Có thể bê sang dự án khác mà không kéo theo cả thế giới.

Quan hệ giữa Cohesion-Coupling và SOLID

Năm nguyên lý SOLID đều quy về cohesion cao + coupling thấp ở các góc nhìn khác nhau:

SOLID	Cohesion	Coupling	Góc nhìn
SRP	□	,	Mỗi module một mục đích
OCP	,	□	Extend không sửa code cũ
LSP	,	□	Subtype substitutable □ loose coupling
ISP	□	□	Interface nhỏ, client chỉ depend cái cần
DIP	,	□	Depend vào abstraction, không vào concretion

Bạn có thể coi SOLID là *operational rules* cho mục tiêu trừu tượng “high cohesion + low coupling”. SOLID dễ áp dụng hơn vì có rule rõ ràng; cohesion-coupling khó áp dụng trực tiếp vì là khái niệm trừu tượng.

Khi nào “coupling thấp” lại là bẫy?

Cảnh báo: cố giảm coupling đến mức 0 dẫn tới *over-abstraction*. Symptoms:

- Mọi class đều phải có interface riêng (kể cả class chỉ có 1 implementation).
- Dùng dependency injection cho mọi thứ, kể cả utility function (StringFormatter thay vì format(s)).
- Vẽ ra layer abstract mà không có lý do business.

Hệ quả: code đọc khó hơn (phải jump qua nhiều layer), build chậm hơn, performance kém hơn.

Heuristic: chỉ tách ra interface khi:

1. Có thật sự >1 implementation (hoặc rất có khả năng có trong 6 tháng tới).
2. Cần mock cho test.
3. Boundary giữa 2 team / 2 service / 2 deployment.

Đừng tách interface “phòng ngừa”, đó là over-engineering.

Tóm tắt

- **Cohesion** = phần tử trong module tập trung một mục đích □ cao là tốt.
- **Coupling** = ràng buộc giữa các module □ thấp là tốt.
- Mỗi cái có 6-7 mức, gom được thành 3 mức (thấp/trung/cao).
- Mục tiêu vĩnh cửu: **high cohesion + low coupling**.
- SOLID là operational rules để đạt mục tiêu này.
- Cần thận over-engineering khi cố giảm coupling.

Bài tiếp: [SRP, Single Responsibility Principle](#), nguyên lý đầu tiên trong SOLID và thường bị hiểu sai nhất.

2.3 SRP, Single Responsibility Principle

Tóm tắt một dòng: SRP nói “Mỗi module nên có duy nhất một lý do để thay đổi”, với “lý do” là một *actor* (một nhóm stakeholder có cùng yêu cầu), không phải là “một function” hay “một concept” như thường bị giảng sai.

Nếu bạn đến từ Data Science

SRP rất dễ hiểu qua một script training. Nếu function `train_model()` vừa đọc dữ liệu, vừa tạo feature, vừa train, vừa evaluate, vừa ghi artifact lên registry, function đó có quá nhiều lý do để thay đổi. Data source đổi cũng sửa nó. Feature đổi cũng sửa nó. Metric đổi cũng sửa nó. Registry đổi cũng sửa nó.

SRP gợi ý tách theo actor hoặc lý do thay đổi: data engineer quan tâm ingestion, data scientist quan tâm feature/model/metric, ML engineer quan tâm registry/serving, platform engineer quan tâm deployment/monitoring. Khi mỗi module chỉ phục vụ một nhóm thay đổi chính, code production dễ evolve hơn.

Phiên bản hay bị giảng sai

Mở Google search “Single Responsibility Principle” và bạn sẽ thấy 90% bài viết định nghĩa như sau:

“A class should do only one thing.”

Định nghĩa này gần như vô dụng vì “one thing” có thể là bất cứ scope nào. Một class `User` có method `getName()` và `getEmail()`, hai method, hai “thing”. Vi phạm SRP? Không. Một class `Calculator` có 10 method tính toán khác nhau, vi phạm? Không.

Vấn đề: định nghĩa này không cho bạn rule kiểm tra được. Nó dẫn tới hệ quả tai hại: developer chia class quá nhỏ (“god classes” thành “fragment classes”), code phình ra hàng trăm class siêu nhỏ, gây overhead cognitive cao mà không giảm bug.

Robert C. Martin nhận ra vấn đề này và sau ~20 năm đã định nghĩa lại SRP trong sách *Clean Architecture* (2017):

“A module should be responsible to one, and only one, actor.”

Định nghĩa modern: SRP về Actor

Actor = một nhóm stakeholder có cùng yêu cầu thay đổi đối với module.

Ví dụ trong một hệ payroll:

- **CFO** (Chief Financial Officer) muốn report về cost lương theo phòng ban.
- **HR Manager** muốn export danh sách lương để gửi ngân hàng.
- **DBA** muốn migrate database schema.

Cả ba đều có thể yêu cầu “sửa class `Employee`”, nhưng họ là *ba actor khác nhau*, yêu cầu của họ có thể *xung đột*. CFO muốn thay đổi cách tính cost (vd: phân bổ benefit theo phòng); HR muốn thay đổi format export; DBA muốn thay đổi cách lưu data. Nếu class `Employee` chứa cả ba logic này, ba yêu cầu có thể đụng nhau trong cùng một method.

SRP nói: **mỗi class nên phục vụ đúng một actor**. Có 3 actor $\square$ cần 3 class.

Ví dụ kinh điển

Vi phạm SRP

```
class Employee:
    def __init__(self, name, salary, hours_worked):
        self.name = name
        self.salary = salary
        self.hours_worked = hours_worked

    def calculate_pay(self) -> Money:
        """Dùng bởi accounting team (Actor: CFO)."""
        # Logic phức tạp tính lương theo tax rule mới
        ...

    def report_hours(self) -> Hours:
        """Dùng bởi HR team (Actor: HR Manager)."""
        # Có thể overlap với calculate_pay vì cùng dùng hours_worked
        ...

    def save(self) -> None:
        """Dùng bởi DBA (Actor: tech ops)."""
        # SQL persistence
        ...
```

Ba method, ba actor. Vấn đề:

1. **Bug do shared state:** CFO yêu cầu “tính lương theo gross hours” và HR yêu cầu “report theo net hours”. Cả hai method dùng `self.hours_worked`, đổi một bên dễ phá bên kia.
2. **Merge conflict:** Team CFO sửa `calculate_pay`, team HR sửa `report_hours`, cùng file □ merge conflict.
3. **Test phức tạp:** test `calculate_pay` cần setup state mà `report_hours` cũng dùng.

Tuân thủ SRP

Tách thành 3 class theo 3 actor:

```
@dataclass
class EmployeeData:
    """Pure data, không có business logic."""
    id: str
    name: str
    base_salary: Money
    hours_worked: Hours

class PayCalculator:
    """Actor: CFO. Phụ trách logic tính lương."""
    def __init__(self, tax_rules: TaxRules):
        self._tax = tax_rules
```

```
def calculate(self, emp: EmployeeData) -> Money:
    gross = emp.base_salary + self._overtime_pay(emp.hours_worked)
    return gross - self._tax.compute(gross)

def _overtime_pay(self, h: Hours) -> Money: ...

class HourReporter:
    """Actor: HR Manager. Phụ trách logic report giờ làm."""
    def report(self, emp: EmployeeData) -> HoursReport:
        # Có thể có rule khác (vd: trừ break time) khác calculate_pay
        ...

class EmployeeRepository:
    """Actor: DBA. Phụ trách persistence."""
    def save(self, emp: EmployeeData) -> None: ...
    def find_by_id(self, id: str) -> EmployeeData: ...
```

Bây giờ:

- CFO yêu cầu đổi cách tính lương □ chỉ sửa PayCalculator. HR và DBA không bị ảnh hưởng.
- HR yêu cầu đổi report format □ chỉ sửa HourReporter.
- DBA muốn migrate schema □ chỉ sửa EmployeeRepository. Business logic không đụng.

Cách identify actor trong dự án

Trong dự án thật, “actor” không phải lúc nào cũng rõ ràng. Vài heuristic:

Heuristic 1: Hỏi “ai sẽ yêu cầu sửa cái này?”

Mở một class, đọc từng method, tự hỏi: “Method này, nếu phải sửa, thường vì ai/team nào yêu cầu?”. Nếu các method trong class đến từ 2-3+ team khác nhau □ vi phạm SRP.

Heuristic 2: Map class lên org chart

Một class tuân thủ SRP thường ánh xạ 1-1 lên một role/team trong tổ chức. OrderProcessor phục vụ Order team. PaymentGateway phục vụ Payment team. AuditLogger phục vụ Compliance team.

Nếu một class kéo nhiều phòng ban □ cần tách.

Heuristic 3: Theo dõi history

Tools như git log -p path/to/file.py cho thấy ai đã sửa file này và vì sao (qua commit message). Nếu một file được sửa bởi 5-10 team khác nhau với mục đích khác nhau □ SRP violation.

Heuristic 4: Theo dõi merge conflict

Nếu một file thường xuyên gây merge conflict giữa các team □ đó là dấu hiệu nó đang phục vụ nhiều actor.

Các pattern fix SRP violation

Pattern 1: Extract Class

Tách methods ra một class mới, mỗi class một actor.

Trước:

```
class User:
    def get_profile(self): ...      # UI team
    def calculate_tax(self): ...    # Finance team
    def export_csv(self): ...      # Data team
```

Sau:

```
class User: ...                  # data class
class UserProfile:
    def get(self, user: User): ... # UI team
class TaxCalculator:
    def for_user(self, user: User): ... # Finance team
class UserExporter:
    def to_csv(self, user: User): ... # Data team
```

Pattern 2: Facade pattern

Nếu khách hàng đang quen gọi `User.calculate_tax()` thì việc tách ra ngay sẽ phá API. Dùng Facade để giữ API trong khi tách internal:

```
class User:
    def __init__(self):
        self._profile = UserProfile()
        self._tax = TaxCalculator()
        self._exporter = UserExporter()

    def calculate_tax(self): # legacy API
        return self._tax.for_user(self)
```

Sau đó migrate dần các caller sang `TaxCalculator.for_user(user)` trực tiếp.

Pattern 3: Strategy pattern (cho variation theo actor)

Khi business rule giống cho nhiều entity nhưng *cách* thực hiện khác theo region/customer-type:

```
class TaxCalculator:
    def __init__(self, strategy: TaxStrategy):
        self._strategy = strategy

class VietnamTaxStrategy(TaxStrategy): ...
class UsaTaxStrategy(TaxStrategy): ...
```

CFO Vietnam và CFO USA là 2 actor, mỗi actor có strategy riêng. `TaxCalculator` không bị bloated.

Khi nào SRP là over-engineering

SRP có giá. Trước khi tách class, hỏi 3 câu:

Câu 1: Có thực sự 2+ actor không?

Nếu một class `Calculator` thuộc 100% domain tính toán, dùng bởi 1 team, không cần tách dù có 20 method.

Câu 2: Lifespan dự án bao lâu?

Code prototype 1 tuần đi demo □ không cần SRP. Code production 5 năm □ bắt buộc.

Câu 3: Có merge conflict không?

Nếu một file 6 tháng chưa có conflict, chưa cần tách dù về lý thuyết có 2 actor.

Nguyên tắc: **SRP fix khi pain xuất hiện**, không refactor “phòng ngừa”.

Sai lầm thường gặp

Sai lầm 1: “Một method = một class”

Vi phạm phía ngược. Tách quá nhỏ thành “fragment class” làm code khó đọc:

```
class UserNameGetter:
    def get(self, user): return user.name

class UserAgeCalculator:
    def calculate(self, user): return ...
```

Đó không phải SRP, đó là over-decomposition.

Sai lầm 2: Coi “module” = “class”

SRP áp dụng cho mọi unit of code: function, class, module, package, service. Một function 200 dòng làm 5 việc cũng vi phạm SRP dù không phải class.

Sai lầm 3: Coi “responsibility” = “task”

Một class có thể làm nhiều task (method) miễn là cùng cho một actor. `TaxCalculator` có thể có `calculate_income_tax()`, `calculate_vat()`, `calculate_corporate_tax()`, tất cả cho actor CFO/Finance, không vi phạm SRP.

Sai lầm 4: Bỏ qua context

SRP đúng cho enterprise code không có nghĩa đúng cho startup MVP. Đừng máy móc.

SRP ở mức Architecture

SRP scale lên thành nguyên lý kiến trúc:

- **Microservice = một service một bounded context (actor)**. Đây chính là SRP ở mức service.

- **Module trong monolith = một module một domain.** Cùng ý.
- **Database table = một table một entity.** Tách thành nhiều table khi có nhiều aspect độc lập (vd: tách users và user_profiles nếu chúng evolve khác nhau).

Phần 6 (Microservices) sẽ build trực tiếp trên insight này.

Tóm tắt

- SRP **không phải** “mỗi class một việc”. Định nghĩa modern: “Mỗi module một actor”.
- Identify actor bằng cách hỏi *ai yêu cầu sửa*, theo dõi merge conflict, hoặc map lên org chart.
- Fix violation bằng Extract Class, Facade, hoặc Strategy.
- Cẩn thận over-engineering: tách khi có pain, không tách phòng ngừa.
- Scale lên architecture: microservice = SRP ở mức service.

Bài tiếp: [OCP, Open-Closed Principle](#), về cách mở rộng code mà không sửa code cũ.

2.4 OCP, Open-Closed Principle

Tóm tắt một dòng: OCP nói code đã viết và tested xong không nên phải sửa khi requirement mới đến, chỉ nên *thêm code mới*. Cách phổ biến: thiết kế các điểm extension bằng interface/abstraction, mở rộng qua polymorphism hoặc plug-in.

Nếu bạn đến từ Data Science

OCP xuất hiện khi bạn muốn thêm model mới mà không sửa toàn bộ pipeline. Hôm nay bạn dùng logistic regression, tuần sau thử XGBoost, tháng sau thử neural network. Nếu mỗi lần thêm model bạn phải sửa evaluator, trainer, registry và serving code, hệ thống chưa closed for modification.

Một thiết kế tốt định nghĩa contract chung, ví dụ `fit`, `predict`, `predict_proba`, `metadata`. Model mới chỉ cần implement contract đó. Pipeline không cần biết bên trong là sklearn, LightGBM hay PyTorch. Đây là OCP ở dạng rất thực tế cho ML.

Định nghĩa

Bertrand Meyer phát biểu OCP lần đầu năm 1988:

“Software entities (classes, modules, functions, etc.) should be open for extension, but closed for modification.”

Diễn đạt lại:

- **Open for extension:** Module có thể thêm behavior mới (vd: support thêm format, thêm payment method, thêm reporting engine).
- **Closed for modification:** Khi thêm behavior mới, *không* phải sửa code đã có (đã test xong, đã deploy).

Mới đầu OCP nghe có vẻ paradox: làm sao thêm behavior mà không sửa code? Trả lời: dùng *abstraction* để định nghĩa điểm extension trước, sau đó implement nhiều phiên bản. Code dùng abstraction sẽ không phải sửa khi có implementation mới.

Ví dụ thuyết phục

Vi phạm OCP

Hãy xét hệ tính chiết khấu cho đơn hàng. Đầu tiên có 2 loại khách hàng:

```
class DiscountCalculator:
    def calculate(self, order: Order) -> Money:
        if order.customer_type == 'regular':
            return order.total * Decimal('0.05')
        elif order.customer_type == 'vip':
            return order.total * Decimal('0.15')
        else:
            return Money(0)
```

Một tháng sau, sếp yêu cầu thêm loại 'premium' với 10% discount. Code:

```
def calculate(self, order: Order) -> Money:
    if order.customer_type == 'regular':
        return order.total * Decimal('0.05')
    elif order.customer_type == 'vip':
        return order.total * Decimal('0.15')
```

```
elif order.customer_type == 'premium':
    return order.total * Decimal('0.10')
else:
    return Money(0)
```

Hai tháng sau, sếp thêm 'corporate' với rule phức tạp (depend vào volume). Bốn tháng sau, sếp thêm 'student' với rule depend vào ngày trong tuần. Cuối năm `calculate()` trở thành 200 dòng if-elif chain với bug khắp nơi.

Vấn đề: Mỗi requirement mới đều phải sửa `DiscountCalculator.calculate()`. Code đã test xong cho 'regular' và 'vip' liên tục bị đụng đến, risk regression cao. Đây là OCP violation kinh điển.

Tuân thủ OCP

Tách bằng abstraction:

```
from abc import ABC, abstractmethod

class DiscountStrategy(ABC):
    @abstractmethod
    def calculate(self, order: Order) -> Money: ...

class RegularDiscount(DiscountStrategy):
    def calculate(self, order: Order) -> Money:
        return order.total * Decimal('0.05')

class VipDiscount(DiscountStrategy):
    def calculate(self, order: Order) -> Money:
        return order.total * Decimal('0.15')

class DiscountCalculator:
    def __init__(self, strategies: dict[str, DiscountStrategy]):
        self._strategies = strategies

    def calculate(self, order: Order) -> Money:
        strategy = self._strategies.get(order.customer_type)
        return strategy.calculate(order) if strategy else Money(0)
```

Bây giờ thêm 'premium':

```
class PremiumDiscount(DiscountStrategy):
    def calculate(self, order: Order) -> Money:
        return order.total * Decimal('0.10')

# Wire up
calculator = DiscountCalculator({
    'regular': RegularDiscount(),
    'vip': VipDiscount(),
    'premium': PremiumDiscount(), # chỉ ADD, không MODIFY
})
```

- RegularDiscount, VipDiscount, DiscountCalculator **không bị sửa** dòng nào.
- Thêm 'premium' = thêm 1 file mới + wire up. Code đã test vẫn nguyên.

Đây là OCP: **closed for modification** (3 file gốc), **open for extension** (thêm file mới).

Cơ chế thường dùng

1. Polymorphism qua interface/abstract class

Như ví dụ trên, pattern phổ biến nhất, đặc biệt cho variation theo type.

2. Strategy pattern

Khi behavior phụ thuộc context. Vd: PriceCalculator có strategy RegularPricing, SeasonalPricing, FlashSalePricing.

3. Plugin architecture (Microkernel)

Scale OCP lên architecture level. Vd: VS Code có core nhỏ + hàng nghìn plugin. Adding feature = adding plugin, không sửa core. Bài 5.5 sẽ đi sâu.

4. Hook / Callback

Cho framework. Vd: WordPress có hook (action, filter); React có lifecycle hooks. Framework không biết business logic, user code "extend" thông qua hook.

5. Configuration

Đôi khi rule phức tạp có thể move ra config:

```
DISCOUNT_RULES = {
    'regular': {'rate': 0.05},
    'vip': {'rate': 0.15, 'min_order': 1_000_000},
    'corporate': {'rate': 0.10, 'volume_breaks': [...]},
}
```

Thêm customer type mới = sửa config (không sửa Python code). Đây là OCP với cost rất thấp khi rule không phức tạp.

OCP ở mức Architecture

OCP scale up thành các pattern kiến trúc lớn:

Microkernel (Plug-in) Architecture

Core hệ thống stable, được lock down. Mọi feature mới là plug-in extend qua well-defined interface.

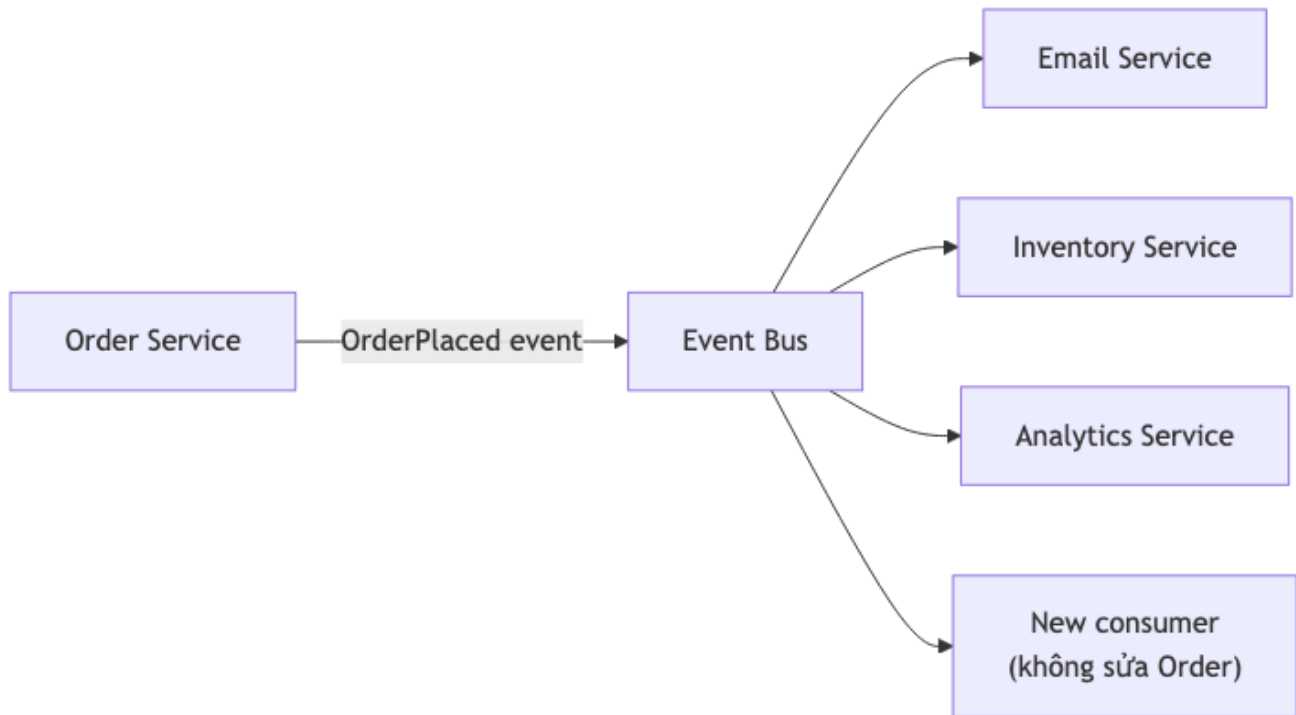
Ví dụ:

- **IDE**: Eclipse, VS Code, IntelliJ, core editor + plug-in ngôn ngữ.
- **Browser**: Chrome, core engine + extension.
- **CMS**: WordPress, core + theme + plug-in.

Bài 5.5 sẽ đi sâu.

Event-Driven Architecture

Producer emit event không biết ai consume. Adding consumer mới = adding một service mới subscribe vào topic, không sửa producer.



Hình 1: Sơ đồ 2

Bài 6.4 sẽ đi sâu.

API Versioning

Khi API cần evolve nhưng không thể break existing clients. Pattern: hỗ trợ song song /v1/... và /v2/.... Adding feature = thêm /v2/, không sửa /v1/. OCP at API level.

Configuration-driven systems

Hệ thống nơi behavior được điều khiển bởi config. Adding feature = ship config mới mà không deploy lại app. Feature flag là một dạng đơn giản của pattern này.

Khi nào OCP là over-engineering

OCP đắt. Mỗi điểm extension cần:

- Một abstraction (interface, abstract class).
- Một registry / factory để wire concrete vào.
- Documentation về contract của abstraction.
- Test cho both abstraction và concrete.

Cost này chỉ đáng khi:

1. **Predict được variation point.** Bạn biết “tương lai sẽ thêm nhiều loại discount” □ OCP cho discount.
2. **Variation đã xảy ra ≥ 2 lần.** “Rule of three” của Martin Fowler: lần đầu hardcoded, lần thứ 2 vẫn hardcoded (nhưng để ý), lần thứ 3 mới refactor sang OCP.
3. **Cost-of-modification cao.** Code legacy sửa nguy hiểm □ đầu tư OCP để future change không sửa.

Đừng OCP “phòng ngừa”. Hệ quả tiêu cực:

- Code phức tạp hơn cần thiết (abstraction không có nhiều implementation).
- Developer mới khó đọc (phải jump qua abstraction).
- Performance kém hơn (virtual call, indirection).

YAGNI (You Aren't Gonna Need It) là counter-principle quan trọng. Cân bằng OCP với YAGNI theo context.

Sai lầm thường gặp

Sai lầm 1: Coi mọi if-elif là OCP violation

Sai. If-elif vẫn ổn nếu:

- Số nhánh ít và ổn định (vd: 3 trạng thái của state machine không bao giờ đổi).
- Logic ngắn, không depend external.
- Không phát triển thêm trong foreseeable future.

OCP fix khi: nhánh nhiều, hay thêm, mỗi nhánh logic phức tạp.

Sai lầm 2: Abstract mọi class

“Phòng ngừa”: tạo interface cho mọi class kể cả class chỉ có 1 implementation. Hệ quả: code đầy IFooImpl mà nội dung interface = nội dung class.

Quy tắc: chỉ tạo interface khi cần ≥ 2 implementation hoặc cần mock cho test.

Sai lầm 3: Quên invariant của abstraction

Khi định nghĩa abstraction, không nói rõ contract:

```
class PaymentStrategy(ABC):
    @abstractmethod
    def charge(self, amount: Money) -> PaymentResult: ...
```

Question: charge có throw exception không? Có retry tự động không? Có idempotent không? Nếu không nói rõ, mỗi implementation có thể có behavior khác □ vi phạm LSP (xem Bài 2.5).

Sai lầm 4: OCP làm hỏng cohesion

Khi tách quá nhiều abstraction, code rời rạc đến mức không hiểu flow. Đọc một use case phải jump qua 5 file. Đây là dấu hiệu over-OCP, cohesion giảm để cố tăng extensibility.

Tóm tắt

- OCP: **open for extension, closed for modification.**
- Đạt qua: polymorphism, strategy, plug-in, event, config-driven.

- Scale lên architecture: Microkernel, Event-Driven, API versioning.
- Cần thận: OCP đắt, áp dụng khi có variation thực tế.
- Cân bằng với YAGNI: rule of three.

Bài tiếp: [LSP, Liskov Substitution Principle](#), về cách inheritance phải tuân thủ contract.

2.5 LSP, Liskov Substitution Principle

Tóm tắt một dòng: Nếu code đang dùng kiểu T, bạn pass vào instance của subtype S (kế thừa T) thì code phải vẫn chạy đúng, nếu không, S không xứng là subtype của T dù compiler chấp nhận.

Nếu bạn đến từ Data Science

LSP có thể hiểu qua sklearn estimator. Khi bạn viết code dùng `model.predict(X)`, bạn kỳ vọng bất kỳ model nào thay vào cũng trả về số lượng prediction khớp với số dòng input, không mutate X bất ngờ, và không yêu cầu caller biết chi tiết nội bộ. Nếu một model con phá kỳ vọng đó, nó không substitutable.

Trong production, LSP giúp serving code an toàn hơn. Nếu mọi model version tuân thủ cùng inference contract, bạn có thể deploy model mới mà không sửa API layer. Nếu contract bị phá, lỗi thường xuất hiện ở runtime, đúng lúc user thật đang gọi hệ thống.

Phát biểu gốc

Barbara Liskov (Turing Award 2008) phát biểu năm 1987:

“If S is a subtype of T, then objects of type T may be replaced with objects of type S without altering any of the desirable properties of the program.”

Diễn đạt programmer-friendly: nếu mọi chỗ code đang dùng T, bạn nhét vào subclass S thì *không có gì hỏng*. Client code không cần biết T thực sự là S hay T thuần.

LSP nghe tự nhiên, ai cũng nói “tất nhiên rồi”. Nhưng thực tế programmer vi phạm LSP suốt mà không nhận ra.

Ví dụ kinh điển: Square is-a Rectangle?

Toán học nói “hình vuông là một hình chữ nhật” (cả hai góc vuông, hình vuông chỉ là special case khi 4 cạnh bằng nhau). Khi code:

```
class Rectangle:
    def __init__(self, width: float, height: float):
        self._width = width
        self._height = height

    def set_width(self, w: float): self._width = w
    def set_height(self, h: float): self._height = h
    def area(self) -> float: return self._width * self._height

class Square(Rectangle): # OOP nói: Square IS-A Rectangle
    def __init__(self, side: float):
        super().__init__(side, side)

    def set_width(self, w: float):
        self._width = w
        self._height = w # phải sync để vẫn là Square

    def set_height(self, h: float):
        self._width = h
        self._height = h
```

Trông hợp lý? Hãy xem client code dùng Rectangle:

```
def expand_to_4x_area(rect: Rectangle):
    """Phóng to rectangle lên 4 lần diện tích bằng cách x2 width."""
    rect.set_width(rect._width * 2)
    rect.set_height(rect._height * 2)
    # Kỳ vọng: area mới = 4 * area cũ

r = Rectangle(3, 5) # area = 15
expand_to_4x_area(r)
assert r.area() == 60 # OK: 6 * 10 = 60

s = Square(4) # area = 16
expand_to_4x_area(s)
assert s.area() == 64 # FAIL! Thực tế là 256
```

Square vi phạm LSP. Tại sao? Vì client `expand_to_4x_area` kỳ vọng *behavior* của `Rectangle`: width và height độc lập. Square phá behavior đó bằng cách sync chúng.

Compiler chấp nhận Square is-a Rectangle về mặt syntax (cùng API), nhưng *contract behavioral* bị phá. Đây là LSP violation.

Cách sửa

Square không phải subtype của Rectangle về behavioral. Sửa bằng cách:

Option A: Bỏ inheritance, tách 2 type độc lập:

```
class Rectangle: ...
class Square: ...
# Nếu cần dùng chung: tạo interface Shape
```

Option B: Tách abstraction immutable (không có setter):

```
@dataclass(frozen=True)
class Rectangle:
    width: float
    height: float
    def area(self) -> float: return self.width * self.height

@dataclass(frozen=True)
class Square(Rectangle):
    def __init__(self, side: float):
        super().__init__(side, side)
```

Vì không có setter, LSP không bị phá. Square thực sự “is-a” Rectangle ở mức immutable.

Contract của abstraction: ba thành phần

Để check LSP rigorous, hãy xét *contract* của method. Contract gồm 3 phần (Bertrand Meyer, Design by Contract):

1. Preconditions

Điều kiện caller phải đảm bảo trước khi gọi. Ví dụ: `sqrt(x)` yêu cầu $x \geq 0$.

LSP rule: Subtype không được *yêu cầu mạnh hơn* (strengthen preconditions). Nếu `Rectangle.set_width` chấp nhận mọi float dương, `Square.set_width` không được yêu cầu thêm điều kiện (vd: $w > 10$).

2. Postconditions

Điều kiện callee đảm bảo sau khi gọi xong. Ví dụ: `sort(arr)` đảm bảo mảng sorted.

LSP rule: Subtype không được *yếu hơn* (weaken postconditions). Nếu `Rectangle.area` đảm bảo `result = width * height`, `Square.area` không được trả về số khác.

3. Invariants

Điều kiện luôn đúng về object. Ví dụ: `BankAccount.balance` ≥ 0 .

LSP rule: Subtype phải *giữ nguyên hoặc strengthen* invariants. Không được nới lỏng (vd: cho phép balance âm cho `OverdraftAccount` mà không tăng cường biến khác).

Liskov's rule chính thức

Subtype S of T phải: - Accept input ít nhất rộng bằng T (precondition không strengthen). - Produce output ít nhất chặt bằng T (postcondition không weaken). - Maintain invariants của T.

Nhớ ngắn: **contravariant inputs, covariant outputs, preserve invariants.**

Ví dụ trong thực tế

V phạm 1: Square / Rectangle (đã xem)

V phạm 2: Penguin / Bird

```
class Bird:
    def fly(self) -> Position: ...

class Penguin(Bird):
    def fly(self) -> Position:
        raise NotImplementedError("Penguins don't fly!")
```

Client gọi `bird.fly()` không expect exception. Penguin strengthen precondition (caller phải check `is_penguin` trước) □ LSP violation.

Fix: tách hierarchy:

```
class Bird: ...
class FlyingBird(Bird):
    def fly(self) -> Position: ...
class Penguin(Bird): ... # không kế thừa FlyingBird
```

Vi phạm 3: Read-only override

```
class List:
    def add(self, item): self._items.append(item)

class ImmutableList(List):
    def add(self, item):
        raise RuntimeError("immutable")
```

Client xài List gọi add() không expect exception. ImmutableList strengthen precondition.

Fix:

```
class ReadOnlyList: # base type không có add
    def get(self, i): ...

class List(ReadOnlyList):
    def add(self, item): ...
```

Client cần modify □ require List. Client chỉ đọc □ require ReadOnlyList. Mỗi loại bị check ở compile time.

Vi phạm 4: ORM “lazy” overriding

```
class User:
    def get_orders(self) -> list[Order]:
        return self._orders # đã load sẵn

class LazyUser(User):
    def get_orders(self) -> list[Order]:
        return self._db.query(...) # query mỗi lần gọi → slow
```

Client gọi user.get_orders() trong loop expect cheap operation. LazyUser weaken postcondition về performance □ vi phạm LSP (về non-functional contract).

Fix: tải liệu hoá rõ “may trigger DB query” trong base class, hoặc tách method load_orders() vs cached_orders().

Code smell báo hiệu LSP violation**Smell 1: if isinstance(x, SubType) trong client code**

```
def process(item: Item):
    if isinstance(item, SpecialItem):
        # logic khác
    else:
        # logic chung
```

Client phải biết subtype □ LSP violation đang giấu. Solution: dùng polymorphism (item.process()).

Smell 2: Override để throw NotImplementedError

Đã thấy ở Penguin.fly().

Smell 3: Override để do-nothing

```
class Base:
    def cleanup(self): self._free_resources()

class Subclass(Base):
    def cleanup(self): pass # do nothing
```

Base contract: “after cleanup, resources freed”. Subclass weaken □ LSP violation.

Smell 4: Tham số override với kiểu hẹp hơn

```
class Animal:
    def feed(self, food: Food): ...

class Cat(Animal):
    def feed(self, food: CatFood): ... # CatFood < Food
```

Cat.feed strengthen precondition. Client xài `animal.feed(generic_food)` sẽ vỡ với Cat.

LSP ở mức Architecture

LSP scale lên thành nguyên tắc API design:

REST API versioning

Khi release `/v2/users`, phải đảm bảo `/v1/users` vẫn behave như cũ. Phá `v1` = vi phạm LSP ở API level. Đây là lý do API versioning quan trọng.

Database migration backward compatibility

Đổi schema không được phá ứng dụng đang chạy (rolling deploy). Add column OK, drop column thì phá LSP với client cũ.

Service interface stability

Microservice A consume API của B. B release version mới phải compatible với A trong period transition. Pattern: expand-and-contract migrations.

Plug-in interface

VS Code core định nghĩa plug-in API. Update core phá API □ mọi plug-in vỡ. Đảm bảo LSP cho plug-in = đảm bảo backward compatibility của API.

Sai lầm thường gặp**Sai lầm 1: Tin compiler đã check LSP**

Compiler check signature (type, arity), không check behavioral contract. Code có thể “type-correct” nhưng vi phạm LSP.

Sai lầm 2: Cõi LSP chỉ là về inheritance

LSP cũng áp dụng cho duck typing (Python, JS), interface implementation, và bất kỳ kiểu polymorphism nào. Bất cứ khi nào client expect behavior nào đó □ ai cung cấp behavior đó phải tuân thủ.

Sai lầm 3: Cố ép is-a relationship khi nó không có

Nhiều khi 2 concept có vẻ liên quan nhưng không phải is-a. Square và Rectangle là ví dụ. Trước khi extend, hỏi: “subclass có thực sự *substitutable* không?”. Nếu chỉ “có vẻ giống” □ dùng composition thay vì inheritance.

Sai lầm 4: Bỏ qua non-functional contract

LSP không chỉ về functional behavior. Performance, memory, network calls cũng là contract. `LazyUser.get_orders()` example.

Tóm tắt

- LSP: **subtype phải substitutable** cho supertype mà không phá client.
- Check qua **contract**: preconditions, postconditions, invariants.
- Rule: contravariant input, covariant output, preserve invariants.
- Smell: `isinstance` check, `NotImplementedError` override, do-nothing override.
- Scale lên architecture: API versioning, schema migration, plug-in stability.
- Cảnh thận: LSP không check được bởi compiler; cần design review + behavioral test.

Bài tiếp: [ISP, Interface Segregation Principle](#), về cách thiết kế interface nhỏ và focused.

2.6 ISP, Interface Segregation Principle

Tóm tắt một dòng: Đừng force client phụ thuộc vào method nó không dùng. Tách interface lớn thành nhiều interface nhỏ và focused, mỗi cái phục vụ một role/use case cụ thể.

Nếu bạn đến từ Data Science

ISP giúp tránh một interface quá to cho mọi loại model. Không phải model nào cũng train online được. Không phải model nào cũng explain được. Không phải model nào cũng support `partial_fit`. Nếu bạn ép mọi model implement một interface khổng lồ, nhiều method sẽ raise `NotImplementedError`, và client code phải check lung tung.

Tách interface nhỏ hơn sẽ tự nhiên hơn: `Predictor`, `Trainer`, `Explainer`, `OnlineLearner`, `BatchScorer`. Client nào cần predict chỉ phụ thuộc `Predictor`. Client nào cần explain mới phụ thuộc `Explainer`.

Định nghĩa

Robert C. Martin, *Agile Software Development* (2002):

“Clients should not be forced to depend upon interfaces that they do not use.”

Diễn đạt lại: nếu interface `IUser` có 20 method nhưng client chỉ dùng 3, client vẫn bị couple với 17 method còn lại. Sửa 17 method đó □ ảnh hưởng client. Đó là coupling không cần thiết.

Ví dụ cổ điển

Vi phạm ISP

```
class Worker:
    def work(self): ...
    def eat(self): ...
    def sleep(self): ...

class HumanWorker(Worker):
    def work(self): print("Working")
    def eat(self): print("Eating")
    def sleep(self): print("Sleeping")

class RobotWorker(Worker):
    def work(self): print("Working")
    def eat(self): raise NotImplementedError # robot không ăn
    def sleep(self): raise NotImplementedError # robot không ngủ
```

Robot bị force implement `eat`, `sleep` dù không cần. Khi `Worker` thêm method (vd: `take_break`), Robot phải sửa nữa.

Client xài `Worker.eat()` không biết Robot sẽ throw □ LSP violation cũng xảy ra (ISP violations thường đi kèm LSP violations).

Tuân thủ ISP

Tách thành interface focused:

```
class Workable:
    def work(self): ...
```



```

class Feedable:
    def eat(self): ...

class Sleepable:
    def sleep(self): ...

class HumanWorker(Workable, Feedable, Sleepable):
    def work(self): ...
    def eat(self): ...
    def sleep(self): ...

class RobotWorker(Workable): # chỉ implement cái cần
    def work(self): ...

```

Client cần làm việc □ require Workable. Client cần feed □ require Feedable. Không có hai bên bị force.

ISP ở mức code

Pattern 1: Role-based interfaces

Đặt tên interface theo *role* (vai trò) chứ không phải theo *type*. Vd:

- Sai: class IUser chứa mọi method liên quan user.
- Đúng: class IAuthenticatable, class IBillable, class INotifiable, mỗi role một interface.

Pattern 2: Capability interfaces

Đặt tên theo *capability* (khả năng):

```

class Readable:
    def read(self) -> bytes: ...

class Writable:
    def write(self, data: bytes) -> None: ...

class Seekable:
    def seek(self, pos: int) -> None: ...

class File(Readable, Writable, Seekable): ...
class ReadOnlyFile(Readable, Seekable): ...
class StreamWriter(Writable): ...

```

Python io module dùng pattern này.

Pattern 3: Tách theo client

Nếu một class phục vụ 2-3 client khác nhau, tách interface cho mỗi client:

```

# Trước (vi phạm ISP)
class UserService:
    # Cho UI:

```

```

def get_display_name(self, id): ...
def get_avatar_url(self, id): ...
# Cho Analytics:
def get_signup_date(self, id): ...
def get_activity_score(self, id): ...
# Cho Admin:
def lock_account(self, id): ...
def reset_password(self, id): ...

# Sau (tuân thủ ISP)
class UserDisplayService:
    def get_display_name(self, id): ...
    def get_avatar_url(self, id): ...

class UserAnalyticsService:
    def get_signup_date(self, id): ...
    def get_activity_score(self, id): ...

class UserAdminService:
    def lock_account(self, id): ...
    def reset_password(self, id): ...

```

UI chỉ depend UserDisplayService, không bị couple với admin/analytics logic.

ISP ở mức Architecture

BFF, Backend for Frontend pattern

Cùng business logic backend phục vụ nhiều client (web, mobile, admin). Mỗi client có *use case khác nhau*:

- Web cần nhiều info trên trang dashboard.
- Mobile cần ít info (bandwidth), nhưng cần thêm push token.
- Admin cần data ẩn cho regular user.

Nếu cùng một API serve cả 3 □ vi phạm ISP ở mức API. Solution: BFF.

Mỗi BFF cung cấp API tailor-made cho client. Core services không cần biết về client.

API Gateway pattern

Tương tự BFF nhưng ở mức gateway. Gateway có thể aggregate nhiều microservice thành response phù hợp client. Mỗi endpoint của gateway = một role-specific interface.

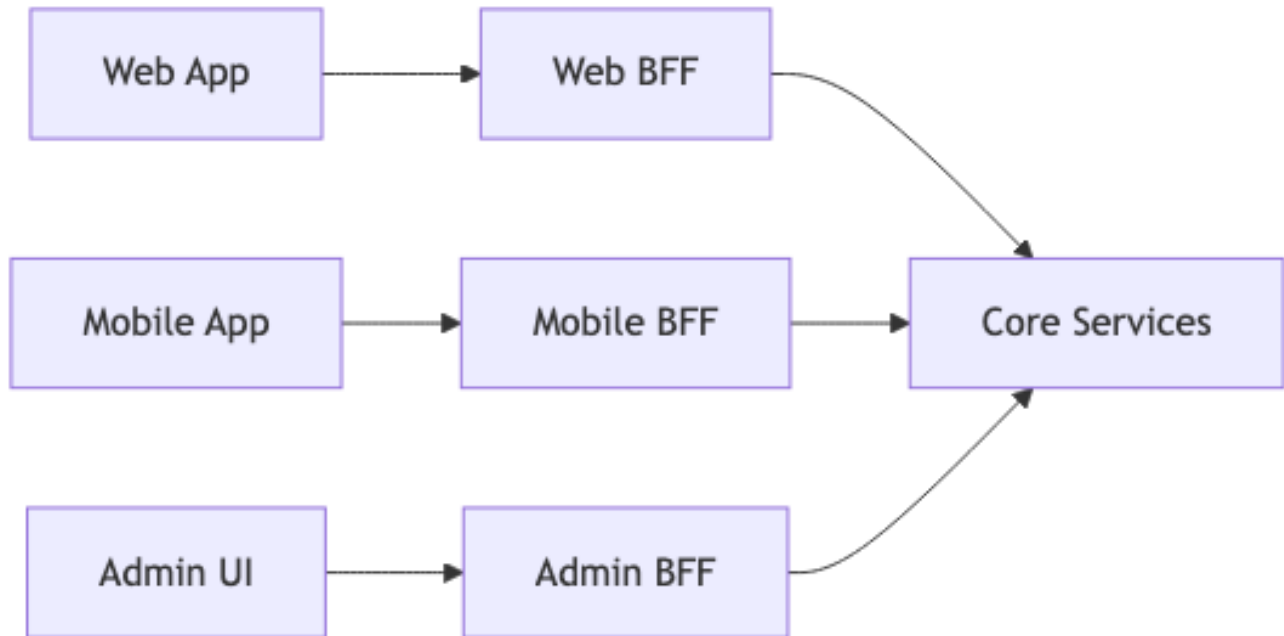
CQRS, Command Query Responsibility Segregation

Tách read API (queries) khỏi write API (commands). Mỗi side có interface riêng:

```

class OrderQueryService: # read-only
    def get_order(self, id): ...
    def list_user_orders(self, user_id): ...
    def search_orders(self, criteria): ...

```



Hình 2: Sơ đồ 3

```

class OrderCommandService: # mutations
    def place_order(self, ...): ...
    def cancel_order(self, id): ...
    def update_shipping(self, id, ...): ...
  
```

UI hiển thị □ chỉ depend OrderQueryService. UI submit form □ depend OrderCommandService. Test, mock, evolve độc lập.

Khi nào ISP là over-engineering

Như mọi nguyên lý SOLID, ISP có cost: nhiều interface = nhiều file = cognitive load.

Cần nhắc:

- **Áp ISP** khi có ≥ 2 client với nhu cầu khác nhau.
- **Bỏ qua** khi interface chỉ phục vụ 1 client (split không có ích).
- **Bỏ qua** khi method trong interface đều có cohesion cao (vd: BankAccount có deposit, withdraw, get_balance, tất cả phục vụ “bank account” role, không tách).

Nguyên tắc: ISP fix khi *client thực sự bị force* depend vào method không dùng. Không “phòng ngừa”.

Sai lầm thường gặp

Sai lầm 1: Single-method interfaces khắp nơi

Java codebase thường có IFooReader, IFooWriter, IFooDeleter, IFooUpdater cho mỗi entity. Đó không phải ISP, đó là over-decomposition. Method có cohesion cao nên gom chung.

Sai lầm 2: Coi ISP đồng nghĩa với “interface nhỏ”

“Nhỏ” không tự động = ISP-compliant. Một interface 1 method vẫn vi phạm ISP nếu method đó force client depend vào abstraction không cần. Ví dụ: Comparable mà object thực sự không có total ordering.

ISP đo theo *client need*, không theo *kích thước interface*.

Sai lầm 3: Quên client là ai

ISP yêu cầu xác định client trước. Trong code mới, “client” có thể chưa tồn tại □ đừng ISP phòng ngừa. Đợi client xuất hiện rồi tách.

Sai lầm 4: Confused với SRP

SRP về *responsibility* (lý do thay đổi). ISP về *interface* (phương thức client gọi). Một class có thể có 1 responsibility (SRP OK) nhưng expose interface to (ISP fail). Hai principle bổ sung nhau.

Tóm tắt

- ISP: **client không bị force depend vào method nó không dùng.**
- Tách interface theo *role* hoặc *capability* hoặc *client*.
- Scale lên architecture: BFF, API Gateway, CQRS.
- Cần thận: ISP đo theo client need, không theo kích thước.

Bài tiếp (cuối cùng SOLID): [DIP, Dependency Inversion Principle](#), nguyên lý quan trọng nhất ở mức architecture.

2.7 DIP, Dependency Inversion Principle

Tóm tắt một dòng: Đảo ngược direction dependency: thay vì module business logic depend trực tiếp vào module infrastructure (database, network), cả hai depend vào abstraction do business logic định nghĩa. Đây là nền tảng cho hexagonal/clean architecture và mọi DI framework.

Nếu bạn đến từ Data Science

DIP nói rằng logic quan trọng không nên phụ thuộc trực tiếp vào chi tiết infrastructure. Ví dụ training pipeline không nên hard-code BigQuery query, S3 path, MLflow client và Slack webhook trong cùng một function. Nếu làm vậy, muốn chạy local test cũng phải có đủ cloud credentials.

Thiết kế tốt đặt abstraction ở giữa: DatasetRepository, ModelRegistry, MetricLogger, Notifier. Production implementation có thể dùng BigQuery, MLflow và Slack. Test implementation có thể dùng local file và in-memory registry. Nhờ vậy business logic của training vẫn ổn định dù tool thay đổi.

Phát biểu

Robert C. Martin (1996):

1. High-level modules should not depend on low-level modules. Both should depend on abstractions.
2. Abstractions should not depend on details. Details should depend on abstractions.

Hai vế bổ sung nhau. Vế 1 nói *direction* của dependency. Vế 2 nói *abstraction là first-class*.

Direction tự nhiên (sai) vs Inverted (đúng)

Direction tự nhiên: top-down

Programmer mới thường viết:

```
# Module business logic
from infrastructure.postgres import PostgresUserRepository # depend low-level

class UserService:
    def __init__(self):
        self._repo = PostgresUserRepository() # cố định

    def register(self, email, password):
        user = User(email, password)
        self._repo.save(user) # gọi trực tiếp
```

UserService (high-level, business logic) depend PostgresUserRepository (low-level, infrastructure). Vấn đề:

1. **Test phức tạp:** muốn test UserService không cần DB ☐ phải mock toàn bộ Postgres connection, transaction.
2. **Đổi DB khó:** muốn migrate sang MongoDB ☐ sửa UserService.
3. **Coupling cao:** 100 service depend vào Postgres ☐ đổi DB = đổi 100 service.

Direction đúng: inverted

```
# abstractions/user_repository.py (BUSINESS layer định nghĩa)
from abc import ABC, abstractmethod

class UserRepository(ABC):
    @abstractmethod
    def save(self, user: User) -> None: ...
    @abstractmethod
    def find_by_email(self, email: str) -> User | None: ...

# domain/user_service.py (BUSINESS, không biết DB)
class UserService:
    def __init__(self, repo: UserRepository): # depend abstraction
        self._repo = repo

    def register(self, email, password):
        user = User(email, password)
        self._repo.save(user)

# infrastructure/postgres_user_repository.py (DETAIL implement abstraction)
class PostgresUserRepository(UserRepository):
    def save(self, user): ...
    def find_by_email(self, email): ...

# Wire up trong main / DI container
service = UserService(repo=PostgresUserRepository())
```

Direction sau:

- UserService depend UserRepository (abstraction).
- PostgresUserRepository cũng depend UserRepository (implement).
- **Cả hai depend abstraction.** Abstraction định nghĩa bởi business layer.

Diagram:

So với direction tự nhiên ($A \square C$), bây giờ $C \square B \square A$. Dependency của high-level đã “đào” về abstraction.

Vì sao gọi là “Dependency Inversion”?

“Inversion” so với traditional layered architecture, nơi dependency luôn flow từ trên xuống dưới (UI $\square$ business $\square$ DB). DIP đảo ngược một phần: low-level detail (DB) bây giờ depend abstraction (do business layer định nghĩa), không phải ngược lại.

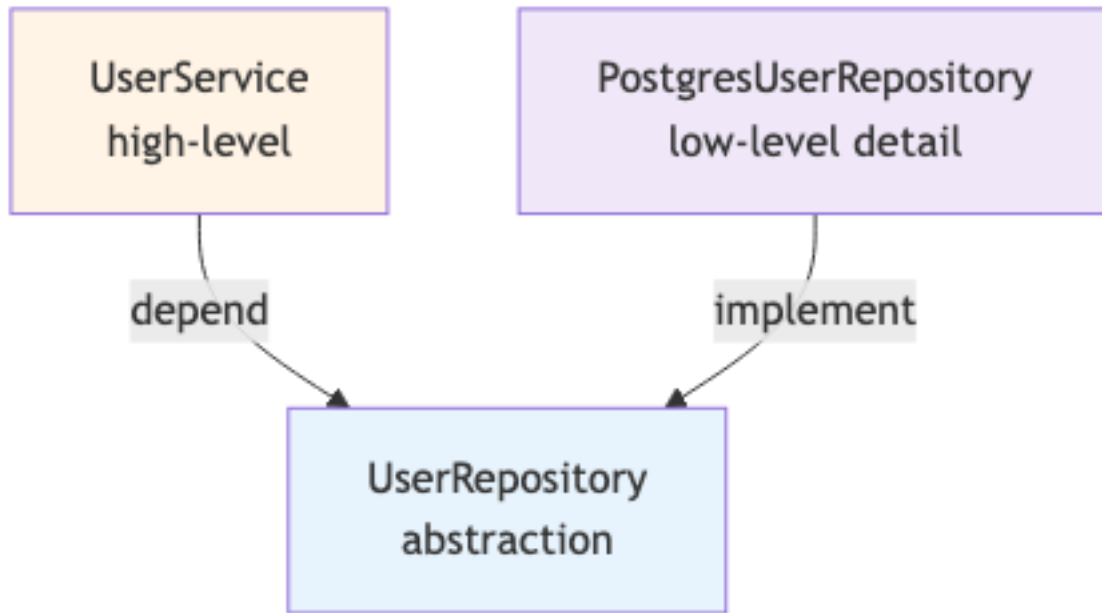
Visual:

Trước (traditional):

```
UI → Business → DB
           □ depend trực tiếp
```

Sau (DIP):

```
UI → Business → IRepository □ DB
```



Hình 3: Sơ đồ 4

□ □
định nghĩa implement

Mũi tên dependency của DB giờ đảo lên trên.

Lợi ích cụ thể

1. Test nhanh, isolated

```

class FakeUserRepository(UserRepository):
    def __init__(self):
        self._users = {}
    def save(self, user): self._users[user.email] = user
    def find_by_email(self, email): return self._users.get(email)

# Test
def test_register_creates_user():
    repo = FakeUserRepository()
    service = UserService(repo=repo)
    service.register("a@b.com", "pass")
    assert repo.find_by_email("a@b.com") is not None
  
```

Không DB, không network, test chạy < 1ms.

2. Đổi infrastructure dễ

Migrate Postgres → MongoDB chỉ cần viết MongoUserRepository(UserRepository). Business layer không sửa.

3. Plug-in architecture

Hệ thống có thể support nhiều backend qua config:

```
def make_repository(backend: str) -> UserRepository:
    if backend == 'postgres': return PostgresUserRepository()
    if backend == 'mongo': return MongoUserRepository()
    if backend == 'memory': return InMemoryUserRepository()
```

4. Parallel development

Team A code business logic + interface. Team B code Postgres impl. Hai team work parallel, ghép qua interface. Không block lẫn nhau.

DIP và Dependency Injection (DI)

Dễ confused. Phân biệt:

- **DIP** = principle (lý thuyết): direction của dependency phải đảo.
- **DI** = technique (thực hành): cung cấp dependency từ ngoài, không tự tạo bên trong class.

DI là *một cách* để đạt DIP, không phải cái duy nhất. Bạn có thể có DI mà không có DIP (vd: inject concrete class), và có DIP mà không có DI (vd: use service locator).

Ba kiểu DI

```
class UserService:
    def __init__(self, repo: UserRepository, mailer: Mailer):
        self._repo = repo
        self._mailer = mailer
```

Constructor injection (tốt nhất) Dependencies rõ ràng trong constructor signature.

```
class UserService:
    def __init__(self, repo): self._repo = repo
    def set_audit_logger(self, logger): self._logger = logger # optional
```

Setter injection (cho optional dependencies)

```
class ReportGenerator:
    def generate(self, data, formatter: Formatter):
        # formatter chỉ dùng ở method này
        return formatter.format(data)
```

Method injection (cho rare-used dependencies)

DI Container / IoC Container

Framework quản lý wiring tự động. Vd: Spring (Java), .NET Core DI, FastAPI Depends, Angular providers.

FastAPI example

```
from fastapi import Depends, FastAPI
```

```
def get_repo() -> UserRepository:
    return PostgresUserRepository()
```

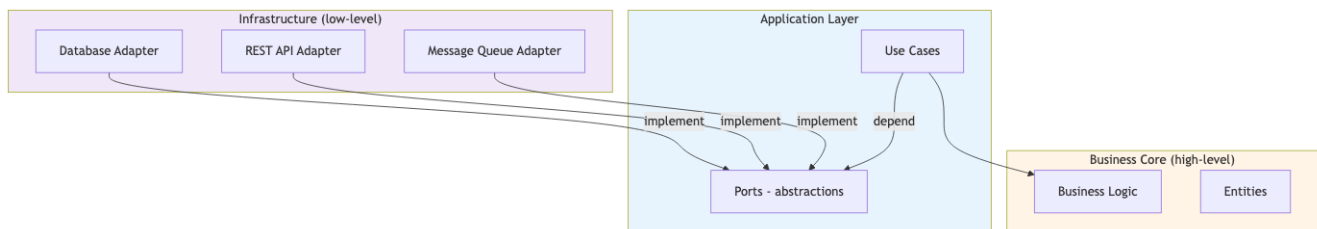
```
@app.post("/users")
```

```
def register(req: RegisterRequest, repo: UserRepository = Depends(get_repo)):
    return UserService(repo).register(req.email, req.password)
```

Lưu ý: DI container không bắt buộc cho DIP. App nhỏ có thể wire manually trong main().

DIP ở mức Architecture**Hexagonal / Clean / Onion Architecture**

Tất cả ba kiến trúc này đều là DIP scaled up. Idea cốt lõi:



Hình 4: Sơ đồ 5

Business core ở giữa (high-level), infrastructure ở ngoài (detail). Direction: detail depend abstraction (port). Business core không biết về infrastructure.

Lợi ích:

- Test business core không cần infrastructure.
- Swap infrastructure dễ (đổi REST $\square$ gRPC, đổi Postgres $\square$ MongoDB).
- Business logic là *stable core*, infrastructure là *volatile shell*.

Microservices boundary

Khi A gọi B qua HTTP/gRPC, A nên define interface (DTO + method signature) mà nó cần. B implement interface đó. Đảo: A không depend trực tiếp implementation của B.

Trong microservices, “abstraction” có thể là OpenAPI spec, Protobuf definition, hoặc message schema. B publish spec, A code theo spec.

Plug-in System

Kiến trúc Microkernel (Bài 5.5) chính là DIP: core định nghĩa plug-in interface, plug-in implement interface. Core không biết về cụ thể plug-in.

Event-driven systems

Producer định nghĩa event schema. Consumer implement handler tuân thủ schema. Producer không biết consumer là ai. Đảo dependency hoàn toàn.

Sai lầm thường gặp

Sai lầm 1: Tạo interface “phòng ngừa” cho mọi class

```
class IUserService: ...
class UserService(IUserService): ...

class IOrderService: ...
class OrderService(IOrderService): ...
```

Mỗi class 1 interface, dù chỉ có 1 implementation. Đó là cargo cult, không lợi ích, chỉ tăng cognitive load.

Quy tắc: interface chỉ tạo khi:

- Có thật ≥ 2 implementation (hoặc rất likely có).
- Cần mock cho test.
- Boundary giữa layer (vd: business vs infrastructure).

Sai lầm 2: Service Locator anti-pattern

```
class UserService:
    def __init__(self):
        self._repo = ServiceLocator.get(UserRepository) # giấu dependency
```

Dependency bị giấu trong implementation, không xuất hiện ở constructor. Test khó (phải setup ServiceLocator). Đọc code khó (không biết class này depend gì).

Constructor injection tốt hơn.

Sai lầm 3: Lạm dụng abstract base class

Trong Python/TypeScript, nhiều khi không cần ABC, duck typing đủ. Tạo ABC chỉ thêm boilerplate.

```
# Quá tay
class UserRepository(ABC):
    @abstractmethod
    def save(self, user): ...

# Vừa đủ (duck typing)
class UserService:
    def __init__(self, repo): # repo: anything with .save method
        self._repo = repo
```

Tradeoff: explicit interface tốt cho large team, duck typing tốt cho code ngắn gọn.

Sai lầm 4: DIP nhưng abstraction leak detail

```
class UserRepository(ABC):
    @abstractmethod
    def execute_sql(self, sql: str): ... # leak SQL
```

`execute_sql` là detail của Postgres, không phải khái niệm business. Repository pattern đúng phải expose business-level method (`save_user`, `find_by_email`), không phải SQL.

Khi abstraction leak detail, swap implementation thành impossible, `MongoUserRepository.execute_sql(sql)` không make sense.

Tóm tắt

- DIP: **high-level và low-level đều depend abstraction**, abstraction do business layer định nghĩa.
- Đạt qua: interface + constructor injection.
- Lợi ích: test dễ, swap infrastructure dễ, plug-in capability.
- Scale lên architecture: Hexagonal/Clean Architecture, plug-in system, event-driven.
- Cảnh thận: không tạo interface phòng ngừa, không service locator, không leak detail qua abstraction.

Tổng kết Phần 2

Hết Phần 2, bạn đã có toàn bộ design principles foundation. Tóm tắt:

Principle	Tóm tắt	Scale up
Cohesion-Coupling SRP	High cohesion, low coupling Mỗi module một actor	Module/service boundary Microservice = một bounded context
OCP	Open extension, closed modification	Plug-in/microkernel architecture
LSP	Subtype substitutable	API versioning, schema migration
ISP	Client không depend method không dùng	BFF, API Gateway, CQRS
DIP	Đảo dependency qua abstraction	Hexagonal/Clean architecture

Tất cả 5 SOLID đều quy về cohesion cao + coupling thấp ở các góc khác nhau. Master 5 cái này, code và architecture của bạn sẽ thay đổi vĩnh viễn.

Phần tiếp theo: [Phần 3: Architectural Thinking](#), mở rộng tư duy từ class lên hệ thống.